

To stride or not to stride the memory access?

Lennart Schmidt
lennart.schmidt1@tu-dresden.de
TU Dresden
Dresden, Germany

Roland Kühn
roland.kuehn@cs.tu-dortmund.de
TU Dortmund University
Dortmund, Germany

Matti Krause
matti.krause@cs.tu-dortmund.de
TU Dortmund University
Dortmund, Germany

Jens Teubner
jens.teubner@cs.tu-dortmund.de
TU Dortmund University
Dortmund, Germany

Wolfgang Lehner
wolfgang.lehner@tu-dresden.de
TU Dresden/AAU Aalborg University
Dresden, Germany

Dirk Habich
dirk.habich@tu-dresden.de
TU Dresden
Dresden, Germany

Abstract

Due to the increasing gap between CPU performance and memory bandwidth, memory access patterns play more and more a significant role for efficient data processing. The current core assumption is that a sequential access pattern delivers the best performance, especially when the data to be processed is stored in adjacent memory locations (contiguous memory). Given the continuous advances in memory technologies, it is of course questionable whether this assumption still holds true. To answer this question, we present a comprehensive experimental comparison of the sequential and the strided access pattern for data stored in contiguous memory on modern disruptive memory systems in this paper. As we are going to show, the core assumption must be revised, as the strided access pattern with a well-chosen stride size clearly outperforms the sequential access pattern. Even a SIMD-accelerated sequential access is considerably slower than the best-performing scalar strided access. In particular, we explain the differences, highlight further advantages, and present open challenges of the strided access pattern on disruptive memory systems.

CCS Concepts: • Information systems → Online analytical processing engines; Query operators; • Hardware → Emerging technologies; Memory and dense storage.

Keywords: Memory Access Pattern, Benchmark, Analysis, High-Bandwidth Memory

ACM Reference Format:

Lennart Schmidt, Roland Kühn, Matti Krause, Jens Teubner, Wolfgang Lehner, and Dirk Habich. 2025. To stride or not to stride the memory access?. In *3rd Workshop on Disruptive Memory Systems (DIMES '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3764862.3768174>



This work is licensed under a Creative Commons Attribution 4.0 International License.

DIMES '25, October 13–16, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2226-4/25/10

<https://doi.org/10.1145/3764862.3768174>

1 Introduction

The key objective of database systems is to reliably manage data, where high query throughput and low query latency are core requirements [1]. To satisfy these requirements, database systems constantly adapt to novel hardware features [2, 4, 5, 8, 10, 14, 16, 18, 24]. Therefore, it is not surprising that current memory developments such as non-uniform memory access (NUMA), high-bandwidth memory (HBM) or remote memory designs such as RDMA or CXL are being intensively researched to exploit their specific properties for an efficient data processing [11, 14, 16, 19]. Moreover, these developments are accompanied by adjustments to the entire memory subsystem to enable an efficient access to data in different memory types. This is particularly noticeable on Intel processors. For example, the 4th generation of the Intel Xeon Scalable processor architecture (Sapphire Rapids) not only offers HBM, but also a significant overhaul of the cache structure in terms of size and associativity compared to previous generations like the older 2nd generation (Cascade Lake). In addition, the 4th generation includes improvements to the structure of the Translation Lookaside Buffer (TLB).

Our Contribution and Outline: These memory subsystem adjustments are very interesting, but the impact on the different memory access patterns is only marginally known [3, 9]. According to the state-of-the-art, it is still assumed that a sequential access pattern provides the best performance, especially if the data to be processed is stored in adjacent memory locations (contiguous memory) [4, 7, 20]. In this paper, we show that this assumption no longer applies for data stored in contiguous memory and that a strided access pattern with a well-chosen stride size clearly outperforms sequential access patterns. A strided access pattern is a variant of the sequential access pattern that realizes an equidistant data access, i.e., there is a constant (but configurable) distance – greater than 1 – between accessed data elements in a contiguous sequence. As data is now retrieved alternately from different non-consecutive positions in memory, it is surprising that this strided access pattern can be more efficient than a sequential access pattern. In the following we will present our insights. The remainder of the paper is structured as follows:

- In Section 2, we present the detailed evaluation setup and the overall results of our comparative evaluation of the sequential and strided access patterns.
- Then, we continue with a profound analysis of the strided access pattern by performing an in-depth investigation of several influencing factors in Section 3. To better analyze individual effects, this evaluation is conducted under a specific setting: a fixed amount of data in local DRAM with a single-threaded execution.
- In Section 4, we expand the scope of our investigation by evaluating and discussing the impact of our obtained insights (i) on varying amounts of data, (ii) on different memory types, such as HBM as well as remote memory, and (iii) on multi-threaded execution.

The paper concludes with related work (Section 5) and a summary, including a broader discussion (Section 6).

2 Access Pattern Comparison

To compare sequential with strided access patterns, we conducted a systematic evaluation of the aggregation sum (AggSum) operation, because the performance of this operation is mainly dominated by the memory performance and therefore the employed access pattern. AggSum computes the aggregating sum over a large column or array of integer data (contiguous memory area). We focused our comparison on the following variants: (1) **sequential**: AggSum will be computed by sequentially iterating over the array and adding up the elements one after the other. (2) **simd-sequential**: This variant is an explicit SIMDified version of the *sequential* variant (using 512-bit AVX512 intrinsics). (3) **strided**: AggSum using a strided access pattern as depicted in Listing 1. The stride size is denoted as s , which also determines the number of required passes over the data (outer loop). In each pass, only every s^{th} array element is accessed (inner loop) with a shifted starting position. For example, with a stride size of 2, two passes over the array are necessary (outer loop). In the first pass, the even array positions (0, 2, 4, ...) are sequentially accessed (half of the elements), while in the second pass, the odd positions (1, 3, 5, ...) are sequentially accessed (other half of the data). (4) **strided-unrolled**: The last variant is an optimized version of the *strided* variant, where the inner loop is explicitly unrolled to reduce the loop overhead as well as to enable greater instruction-level parallelism.

Evaluation Setup. We conducted our evaluation with two generations of Intel Xeon Scalable CPUs, as shown in Table 1. The newer generation (Sapphire Rapids architecture)

Table 1. System specification comparison.

Architecture	Cascade Lake	Sapphire Rapids
Generation	2 nd Xeon Scalable	4 th Xeon Scalable
Name	Gold 6240R	Gold 9468
RAM	DDR4 - 192GB -	DDR5 - 256GB HBM - 64GB
L1D Cache	32KiB (8-way)	48KiB (12-way)
L2 Cache	1MiB (16-way)	2MiB (16-way)
L3 Cache	35.75MiB (11-way)	105MiB (15-way)
DTLB	64 Entries	96 Entries
STLB	1536 Entries	2048 Entries

features 64 GiB of HBM, as well as larger cache sizes and higher cache associativity compared to the older Cascade Lake architecture. Moreover, the Data Translation Lookaside Buffer (DTLB) and the Second-level Translation Lookaside Buffer (STLB) are larger on the newer generation. All AggSum variants were implemented in C++¹ and compiled using g++ (version 14.3.0) with the optimization flags `-O3 -march=native -mavx512f` as well as `-fno-tree-vectorize` to disable autovectorization. All variants were evaluated using different data types (e.g., `uint64_t` or `uint32_t`) and in both single-threaded and multi-threaded settings. In this paper, we only present the results for `uint64_t`, as we observed similar effects for `uint32_t`. All experiments were conducted entirely in-memory with input arrays containing randomly generated values allocated using `aligned_alloc`. We used both standard 4 KiB pages and 2 MiB transparent huge pages. While 4 KiB pages enable a fine-grained memory management, they can burden the TLB with many entries, potentially causing performance issues; huge pages can reduce TLB pressure and improve the performance of memory-intensive operations. All experiments were repeated multiple times with the CPU governor set to *performance* and we report the mean results. Since modern Intel CPUs feature sophisticated hardware data prefetching mechanisms, we do not use software-based prefetching [12, 17].

Evaluation Result A representative excerpt of our single-threaded evaluation results for a 1 GiB array of `uint64_t` values is illustrated in Figure 1. In Figures 1(a) and 1(b), the stride size with a logarithmic scale is shown on the x-axis and the throughput in GiB/s on the y-axis. In our experiments, we explicitly pinned the execution of all AggSum variants on a core of NUMA node 0 and placed the data array in the locally attached DRAM of NUMA node 0 using `numactl`. Since the array contained 2^{27} `uint64_t` elements, we evaluated all possible stride sizes from 1 to 2^{26} in terms of number of elements. From our results, we can derive the following general observations, which are not surprising at all: (i) As expected, *simd-sequential* achieves a higher throughput than *sequential* on both generations. (ii) A strided access with a stride size of 1 (in terms of elements) corresponds to a sequential

```

1 uint64_t* data; /* assume data array is initialized */
2 uint64_t sum = 0;
3 for (size_t i = 0; i < s; i++) {
4     for (size_t j = 0; j < total_elements/s; j++) {
5         sum += data[i + j * s];
6     }
7 }

```

Listing 1. AggSum using a strided access pattern.

¹Code: <https://github.com/db-tu-dresden/DIMES25-AccessEvaluation>

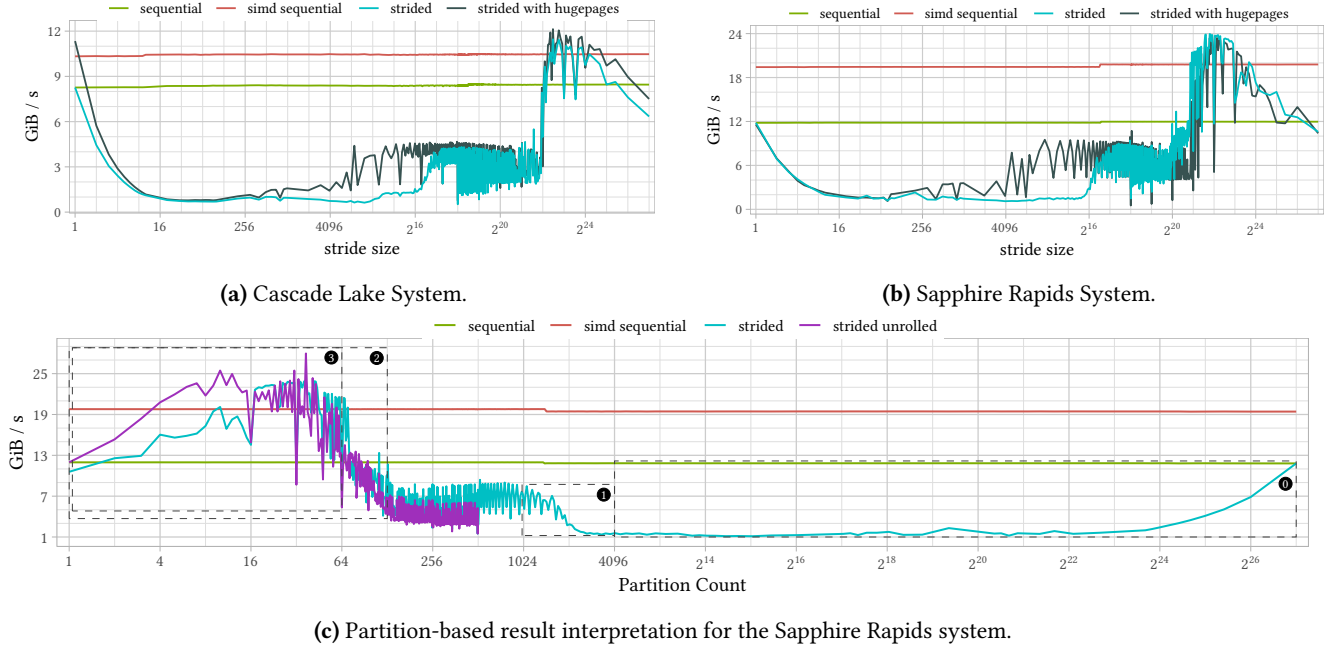


Figure 1. Single-threaded throughput results for AggSum over a 1 GiB array of `uint64_t` values.

access pattern resulting in equal throughput values as visible on both generations. As stride size increases, throughput initially decreases continuously. (iii) For a certain stride size range, huge pages offer performance benefits for the strided access on both generations but not in the interesting area.

However, Figures 1(a) and 1(b) also reveal a *surprising insight*: With increasing large stride sizes (around the stride size of 2^{16}), the throughput of the *strided* variant suddenly increases in several steps, both for 4 KiB and for huge pages. Moreover, the *strided* variant achieves a similar throughput as the *simd-sequential* variant with a stride size of $2^{22.5}$ on the older Cascade Lake architecture on the one hand. On the other hand, the *strided* variant clearly outperforms the *simd-sequential* variant on the newer Sapphire Rapids architecture. While *simd-sequential* achieves a throughput of 19.8 GiB/s, the *strided* variant achieves – in the best case – a much higher throughput of ca. 24 GiB/s for a stride size of $2^{21.8}$.

To explain the throughput behavior of the *strided* variant, we slightly adapt the interpretation: The strided access logically divides the data array into $\frac{n}{s}$ partitions, where (a) n is the total number of array elements and (b) s is the stride size. Thus, each partition consists of exactly s consecutive elements. In the i^{th} strided pass over the data (outer loop in Listing 1), the i^{th} element per partition is processed. Based on that interpretation, Figure 1(c) shows our measured throughput results for the different AggSum variants on the newer Sapphire Rapids architecture. While a small stride size leads to a high partition count, large stride sizes result in small partition counts. As depicted in Figure 1(c), the interesting behavior occurs for small partition counts. Figure 1(c) omits

the curve for strided access using huge pages, as the difference to 4 KiB is marginally in the area of interest, but the figure includes the results for the *strided-unrolled* variant using 4 KiB pages with unrolling only up to a partition count of 512. Again, both strided variants clearly outperform the *sequential* as well as the *simd-sequential* variant for small partition counts. Moreover, it can be seen that unrolling the inner loop of the strided access (cf. Listing 1) for small partition counts provides a significant throughput boost.

3 Micro Analysis

To analyze the throughput behavior of the *strided* variants in detail, we now turn to the observations in Figure 1(c), based on the setup described in Section 2. Generally, the following applies to all access patterns: When an element of the data array is accessed, a cache line is always loaded into the caches, which contains a copy of the memory around this element. A cache line is 64 bytes in size – corresponding to 8 `uint64_t` values – on our Intel (x86) systems. Moreover, if the memory subsystem detects a sequential access pattern, the hardware prefetchers will also prefetch the next cache line (or two) to boost the access to the adjacent data elements.

Based on that, the memory subsystem is hardly stressed by our *sequential* variants, because both variants iterate once over the complete array and all data elements are processed in sequence. This means the memory subsystem is only concerned with the sequential prefetching of neighboring cache lines through the different pages. Then, all elements of the cache lines are aggregated by the CPU and here, the *sequential* and the *simd-sequential* variants differ. While the *sequential* variant executes 8 arithmetic instructions per cache line,

the *simd-sequential* variant only executes one arithmetic SIMD instruction per cache line leading to a higher throughput as shown in Figure 1(c). In contrast, the strided variants heavily stress the memory subsystem as data elements from different non-consecutive positions in memory have to be loaded in a sequential manner. As shown in Figure 1(c), this can have a very positive and a very negative effect on the throughput and this can be explained by four main observations that we have marked and highlighted in Figure 1(c).

Observation 0

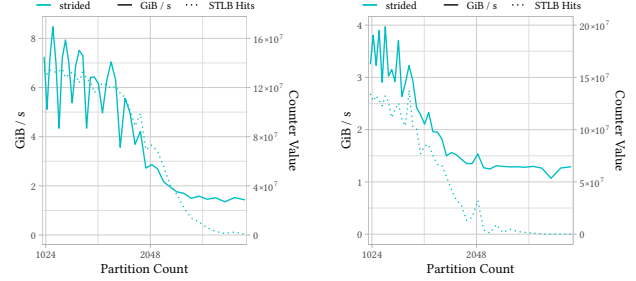
For large numbers of partitions (> 4096), the throughput of the *strided* variant is very low, but smoothly increases for very large numbers of partitions ($> 2^{24}$) until finally reaching the throughput of the *sequential* variant.

The partition count also represents the number of non-consecutive data elements that have to be sequentially loaded per strided pass over the array. For each data element, (i) the virtual address has to be translated into a physical address and (ii) the corresponding cache line has to be fetched. Both tasks overload the memory subsystem for larger partition counts, as each data element requires a disjoint page to be accessed. However, when the number of partitions becomes very large (e. g., $> 2^{24}$ partitions), multiple partitions can then be found within a single page. Furthermore, since our AggSum operates on 1 GiB of data, with 2^{24} partitions, each partition corresponds to exactly one cache line. This means that if the number of partitions further increases, more than one partition will reside within one cache line. Since the partitions are accessed sequentially (see the inner loop of Listing 1), accessing multiple partitions results in multiple accesses to the same cache line. We attribute the increasing throughput with a very large number of partitions (cf. Figure 1(c)) to these effects.

Observation 1

In the partition count range from 1024 to 4096, a first increase in throughput for the strided access can be observed on both examined Intel CPUs.

Influence of the TLB. Figure 2 shows a zoom-in into the throughput curves for the strided AggSum variant within the partition count range from 1024 to 4096 for both examined CPUs. In both diagrams in Figure 2, we can clearly observe a performance increase once the number of partitions is lower than the combined capacity of both TLBs. The point at which the throughput increases occurs at different partition counts on both systems, as the examined systems have different TLB configurations for 4 KiB pages (cf. Table 1). Once the performance increases, we can clearly observe an increase



(a) Sapphire Rapids System.

(b) Cascade Lake System.

Figure 2. Throughput and STLB hits between 1024 and 4096 partitions on both systems.

in STLB hits as well as a decreased number of triggered page walks that fetch the translation from the system's page table. This observation can be confirmed by using huge pages. With page sizes of 2 MiB and 1 GiB, a performance degradation cannot be observed, since the TLB sizes on both systems are now sufficient to hold all required translations. Therefore, we conclude that an important influencing factor for the strided access pattern is the size of the TLB.

Take away: When employing a strided access pattern, the configuration and the capacity of the TLB determine the maximum number of partitions that can be used without incurring performance degradations.

Observation 2

A second significant increase in throughput on our systems can be observed for a small number of partitions (< 64), which even exceeds the throughput of the *simd-sequential* variant.

Influence of Hardware Prefetchers We attribute the observed throughput increase primarily to the advanced hardware prefetching mechanisms. Modern CPU architectures are typically equipped with sophisticated prefetching mechanisms embedded into the hardware that detect memory access patterns and proactively load data into the CPU cache before it is explicitly requested. These hardware prefetchers can not only identify simple ascending or descending address patterns but are also capable of tracking multiple independent memory streams simultaneously. Both of our systems feature L1 hardware prefetchers that observe memory accesses and can trigger additional prefetchers embedded in the L2 cache. To better understand their impact on throughput, we conducted a systematic evaluation by selectively enabling and disabling the individual prefetchers at both levels of the cache hierarchy. As shown in Figure 3, turning off the L1 prefetchers has only a minor impact on the overall performance. However, turning off the L2 prefetchers results in a substantial drop in throughput, highlighting

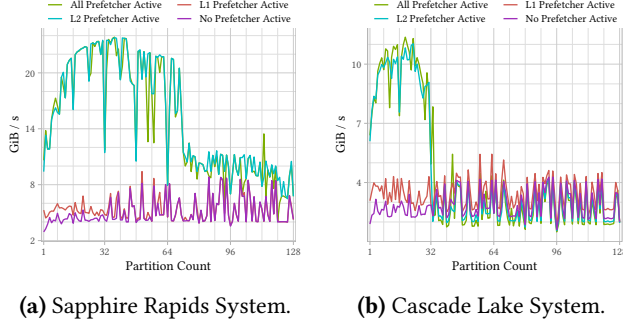


Figure 3. Zoom-in to the partition count range from 1 to 128 with hardware prefetchers enabled and disabled.

their critical role for a strided access. Furthermore, our experiments indicate that the L2 prefetchers seem to track up to 32 concurrent streams on Intel’s older Cascade Lake and up to 72 streams on the newer Sapphire Rapids. Once the number of concurrent non-consecutive memory accesses exceeds this threshold, the prefetcher struggles to maintain its effectiveness, leading to reduced performance.

Take away: The parallel sequential traversal of multiple partitions can lead to significant throughput increases; however, the limits of the hardware prefetchers must be considered, as the number of monitored streams varies between CPUs.

Observation 3

On our systems, downward outliers in throughput were observed for specific partition counts such as 8, 16, 24, ...

Influence of cache associativity We attribute this observation to negative interference with the caching subsystem on the CPU that is caused by reading sequentially from multiple partitions in close temporal proximity. In our experiments, the amount of data is a power of two; therefore, certain partition counts divide the data into multiples of two, which causes the partition starting addresses to share a large number of lower bits. Since caches may use the lower parts of the address for mapping the cache line to its sets in the cache, the partitions all map their cache lines to one or only a few number of cache sets, thereby thrashing the cache even though there is still free space in the caches. To reduce the alignment of the partitions, a 576 byte padding between the partitions was introduced. As the results in Figure 4 show, this measure is effective in reducing or even mitigating the outliers at multiples of 8, but it also introduces new outliers and retains certain old ones. The shown correlation between throughput and L1 correlated stalls substantiates the cache associativity hypothesis, though further investigation is required to better predict the performance based on partition alignment and to verify whether cache associativity is truly the sole cause.

Take away: Data structures with multiple partitions that are expected to be traversed sequentially at the same time,

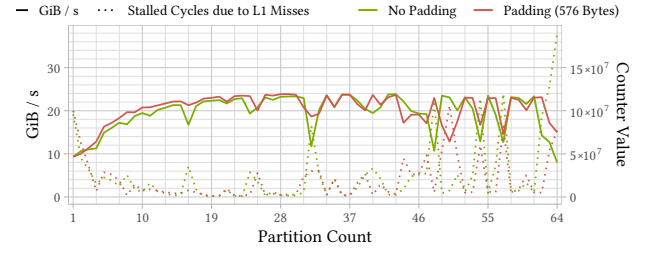


Figure 4. Impact of a fixed padding (576 Bytes) between partitions on the Sapphire Rapids system.

such as column-major tables and struct-of-array constructs, should take care to disalign the starting addresses of their partitions to a reasonable degree to avoid the negative effects of cache associativity.

4 Macro Analysis

In the previous section, our analysis was based on the following restrictions: (i) fixed data size of 1 GiB, (ii) data residing in local DRAM, and (iii) the single-threaded execution of all AggSum variants. In this section, we examine how relaxing these restrictions affects the observations made so far.

4.1 Influence of Partition Count and Stride Size

One important limitation – which is shown in Figure 1 – is that we have so far assumed a fixed data size of 1 GiB. Based on this, the stride size always adjusts accordingly when the number of partitions changes. The result of this is that we cannot draw any conclusion about a relationship between partition count and stride size. To address this, we enhanced our experiments by fixing the stride size across multiple partition counts, resulting in varying data amounts between runs. Figure 5(a) depicts representative results for three different fixed stride sizes across a partition count range from 2 to 50. In this figure, we only report the single-threaded throughput values for the strided access variant of AggSum. The following conclusions can be drawn from these results: (i) The partition count for the highest throughput depends on the stride size. For instance, while the highest throughput is achieved for stride sizes of 127 and 128 MiB with 13 partitions, 42 partitions are required for a stride size of 127.32 MiB. (ii) Some stride sizes lead to higher throughput values than others. For example, the highest throughput for a stride size of 127.32 MiB is roughly 24 GiB/s, while only ~ 17 GiB/s is achieved for stride sizes of 127 and 128 MiB.

To better analyze the influence of the stride size, we modified our experiments by fixing the partition count across multiple stride sizes in a second step. Figure 5(b) shows representative results for three partition counts across the stride size range from 127 to 128 MiB. Moreover, we use the partition counts that previously yielded the best throughput (13 and 42), plus a randomly chosen partition count of 33. Again, we only report the throughput of the single-threaded execution for the strided access variant of AggSum. From these

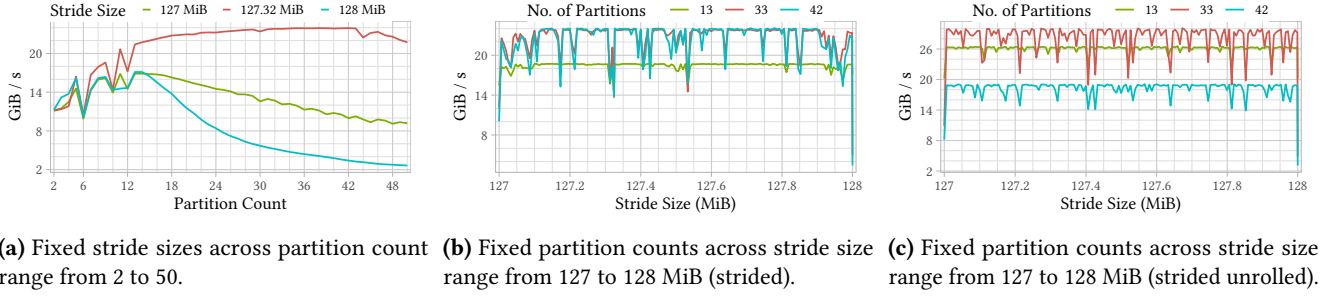


Figure 5. Evaluating influence of partition count and stride size on the Sapphire Rapids system.

results, we are able to draw the following conclusions: (i) For many stride sizes, a high partition count leads to high throughput values. The difference between partition counts of 33 and 42 is often marginal. (ii) However, for some stride sizes, a throughput drop can also be observed, and the number of drops increases with larger partition counts. (iii) The throughput values for stride sizes of 127 and 128 MiB are always by far the lowest.

In the third step, we repeated both experiments for the *strided-unrolled* variant of AggSum. Figure 5(c) shows the obtained throughput results for the same three partition counts across the stride size range from 127 to 128 MiB. When we compare Figures 5(b) and 5(c), we can make the following observations: (i) The partition count of 13 achieves a higher throughput for the *strided-unrolled* variant compared to the *strided* variant. (ii) The partition count of 33 leads to high throughput values in both cases, while the partition count of 42 now delivers low throughput values for the *strided-unrolled* compared to the *strided* variant.

Take away: Our experiments show that an appropriate partition count as well as an appropriate stride size are important factors for achieving a high throughput. However, the specific parameters for a given amount of data cannot be derived from the current results. Moreover, the specific parameters vary for the *strided* and *strided unrolled* variant. For now, two recommendations can be made for our Intel Sapphire Rapids system: (i) The partition count should be between 30 and 42. (ii) The stride size should not be a power of two (128 MiB) or a multiple of a power of two (127 MiB).

4.2 Effects on Emerging Memory Technologies

Modern systems are increasingly complex and offer heterogeneous memory with different latency and throughput properties. This raises the question of whether, and if so in what form, our observed insights on local DRAM can also be observed on remote DRAM – such as in classic multi-socket environments – as well as on modern emerging memory technologies like HBM.

Remote DRAM. In a first step, we examined traditional remote DRAM access, as our Sapphire Rapids system features two sockets. For these experiments, we explicitly pinned the execution of all AggSum variants on a specific core on socket

1 and placed the data array in remote DRAM on socket 2. If we compare the obtained throughput values for remote DRAM with those for local DRAM (cf. Table 2), interesting observations can be made: (1) The remote DRAM throughput for the *sequential* as well as the *simd-sequential* variant is significantly lower than for local DRAM. One reason for this might be that the access latency for remote DRAM is greater than for local DRAM. (2) In contrast, the *strided* variant shows a similar throughput for local and remote DRAM. They only differ by less than 1% in peak performance; a peak throughput of 23.96 GiB/s with 37 partitions on local DRAM and 23.78 GiB/s with 36 partitions on remote DRAM can be achieved. In this case, the memory subsystem appears to compensate very well for the higher latency with this access pattern. However, using loop unrolling, the peak throughput of local DRAM access increases by 4 GiB/s, while no significant changes for remote DRAM can be observed (cf. Table 2). The results for strided access are promising, as local and remote DRAM now nearly the same throughput, unlike sequential access.

We also conducted these experiments on our Cascade Lake system consisting of four sockets. Interestingly, the older generation is more affected by accesses to remote DRAM. Although a very similar throughput curve to that of local DRAM can be observed for remote DRAM, we see performance losses of 30% in peak throughput for all AggSum variants on remote DRAM. This indicates that changes to the Sapphire Rapids memory subsystem require adapting existing strategies for efficient data processing and placement in multi-socket environments [15, 21].

High Bandwidth Memory. In a second step, we also evaluated our different AggSum variants on high-bandwidth memory (HBM). On our Sapphire Rapids system, HBM is exposed as additional NUMA nodes. In addition to evaluations of our AggSum variants on local HBM, we also investigated – similar to the previous paragraph – the effects of accessing remote HBM nodes (i.e., HBM connected to a remote CPU). The obtained results are summarized in Table 2 and we can make the following observations: (1) The throughput for the *sequential* and the *simd-sequential* variants for local HBM is slightly lower than for local DRAM. This was to be expected, as the access latency for HBM is higher than for DRAM. In the most favorable case (*simd-sequential*), the

Table 2. Single-threaded peak throughput results in GiB/s on the Sapphire Rapids system for AggSum over a 1 GiB array on different memory devices.

AggSum variant	DRAM (local)	DRAM (remote)	HBM (local)	HBM (remote)
sequential	11.97	7.86	9.67	5.73
simd-sequential	19.79	11.88	18.79	10.02
strided	23.96	23.78	23.97	9.37
strided-unrolled	27.97	24.07	27.84	9.52

difference is only 1 GiB/s. (2) As for the remote DRAM, the higher access latency can be better hidden by strided access patterns, such that there are only marginal differences in throughput between local DRAM and local HBM. (3) A particularly noteworthy observation can be made when accessing remote HBM: As shown in Table 2, the throughput on remote HBM barely exceeded 10 GiB/s, whereas remote DRAM access achieved over 23 GiB/s. Although strided AggSum variants outperform the *sequential* variant, the *simd-sequential* variant achieves the best throughput on remote HBM. The reason for this is still unknown to us and requires further investigation.

Take away: In our experiments, we observed that, in contrast to previous generations, the Sapphire Rapids architecture does not exhibit major throughput drops for strided accesses to remote DRAM. However, accesses to remote HBM result in substantial performance degradation.

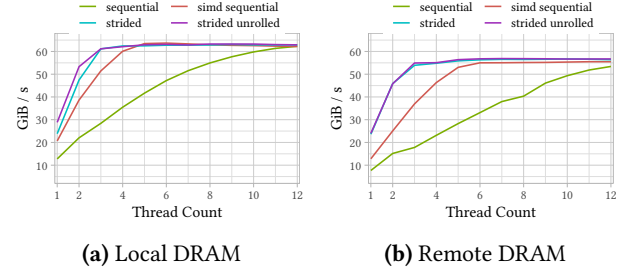
4.3 Effects of Multi-threaded Execution

In order to better isolate and analyze individual effects, the measurements so far have been conducted exclusively single-threaded. In the following, we investigate whether and to what extent the observations made so far can be transferred to a multi-threaded scenario. For this purpose, we measured the different AggSum variants using multiple threads. For the strided variants, the best partition count was selected for each thread count. To ensure that no side effects distorted our results (e.g., due to caching effects), the multi-threaded variants were set up in a way ensuring that each thread processes 1 GiB of data (so for 12 threads, 12 GiB of data were processed). Figure 6 shows—for both local and remote DRAM—the strided variants already achieve a high throughput with far fewer threads compared to the sequential variants.

Take away: With a strided access pattern, fewer threads are needed to fully utilize the available bandwidth, saving energy or freeing resources for other (computational) tasks.

5 Related Work

The most relevant related work is the paper of Blom et al. [3]. In contrast to us, they evaluate more complex operations such as Dense Matrix Vector Multiplication, 3x3 Convolution, or Stencil Computation with different memory access

**Figure 6.** Throughput of the different AggSum variants with increasing number of threads on the Sapphire Rapids system.

patterns. They also conclude that switching from a sequential to a strided access pattern results in a performance boost. However, their comprehensive evaluation is based only on SIMDified implementations using AVX2 intrinsics without a detailed analysis of the stride size. For the SIMDified implementation of a strided access, they use the GATHER instruction and the efficient applicability of this instruction for such a strided access has already been also demonstrated in [9, 23]. In these papers, however, the authors clearly demonstrate that a strided access pattern provides a suitable alternative to SIMD optimization for enhancing the throughput of scalar code that employs a sequential access pattern.

6 Conclusion and Future Work

In this paper, we presented a comprehensive experimental comparison of sequential and strided access patterns for data stored in contiguous memory on two different Intel processor generations. As we have shown, a strided access pattern with a well-chosen stride size clearly outperforms the sequential access pattern. Even a SIMD-accelerated sequential access is considerably slower than the most powerful scalar strided access, as the advanced features of the memory subsystem on the newer Intel processor generation are better utilized. In particular, higher access latencies can be effectively hidden with a strided pattern, leading to an increased throughput.

In this work, we treated the memory subsystem largely as a black box and examined the effects of strided access on two architectures. Our results indicate clear developments in the memory subsystem from which strided access pattern can benefit. However, we are not yet able to provide a formalization for determining an optimal stride size and partition count, since additional factors (e.g., memory-, bank-level, and rank-level parallelism [6, 13, 22]) may also play important roles and require further investigation.

Acknowledgments

This work was partly funded by (1) the German Research Foundation (DFG) priority program SPP 2377 under grant numbers 502444078 (VAMPIR) and 502384507 (Memory Diplomat), and (2) BMBF and the state of North-Rhine Westphalia as part of the Lamarr-Institute for Machine Learning and Artificial Intelligence.

References

- [1] Daniel Abadi, Rakesh Agrawal, Anastasia Ailamaki, Magdalena Balazinska, Philip A. Bernstein, Michael J. Carey, Surajit Chaudhuri, Jeffrey Dean, AnHai Doan, Michael J. Franklin, Johannes Gehrke, Laura M. Haas, Alon Y. Halevy, Joseph M. Hellerstein, Yannis E. Ioannidis, H. V. Jagadish, Donald Kossmann, Samuel Madden, Sharad Mehrotra, Tova Milo, Jeffrey F. Naughton, Raghu Ramakrishnan, Volker Markl, Christopher Olston, Beng Chin Ooi, Christopher Ré, Dan Suciu, Michael Stonebraker, Todd Walter, and Jennifer Widom. 2016. The Beckman report on database research. *Commun. ACM* 59, 2 (2016), 92–99. doi:10.1145/2845915
- [2] André Berthold, Constantin Fürst, Antonia Obersteiner, Lennart Schmidt, Dirk Habich, Wolfgang Lehner, and Horst Schirmeier. 2024. Demystifying Intel Data Streaming Accelerator for In-Memory Data Processing. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems, DIMES 2024, Austin, TX, USA, 3 November 2024*. ACM, 9–16. doi:10.1145/3698783.3699383
- [3] Miguel O. Blom, Kristian F. D. Rietveld, and Rob V. van Nieuwpoort. 2025. Multi-Strided Access Patterns to Boost Hardware Prefetching. In *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering, ICPE 2025, Toronto, ON, Canada, May 5-9, 2025*. ACM, 204–215. doi:10.1145/3676151.3719375
- [4] Peter A. Boncz, Martin L. Kersten, and Stefan Manegold. 2008. Breaking the memory wall in MonetDB. *Commun. ACM* 51, 12 (2008), 77–85. doi:10.1145/1409360.1409380
- [5] Sebastian Breß, Henning Funke, and Jens Teubner. 2016. Robust Query Processing in Co-Processor-accelerated Databases. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*. ACM, 1891–1906. doi:10.1145/2882903.2882936
- [6] Yuan Chou, Brian Fahs, and Santosh G. Abraham. 2004. Microarchitecture Optimizations for Exploiting Memory-Level Parallelism. In *31st International Symposium on Computer Architecture (ISCA 2004), 19-23 June 2004, Munich, Germany*. IEEE Computer Society, 76–89. doi:10.1109/ISCA.2004.1310765
- [7] Andrew Crotty, Alex Galakatos, and Tim Kraska. 2020. Getting Swole: Generating Access-Aware Code with Predicate Pullups. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 1273–1284. doi:10.1109/ICDE48307.2020.00114
- [8] Andreas Geyer, Johannes Pietrzyk, Alexander Krause, Dirk Habich, Wolfgang Lehner, Christian Färber, and Thomas Willhalm. 2023. Near to Far: An Evaluation of Disaggregated Memory for In-Memory Data Processing. In *Proceedings of the 1st Workshop on Disruptive Memory Systems, DIMES 2023, Koblenz, Germany, 23 October 2023*. ACM, 16–22. doi:10.1145/3609308.3625271
- [9] Dirk Habich, Johannes Pietrzyk, Alexander Krause, Juliana Hildebrandt, and Wolfgang Lehner. 2022. To use or not to use the SIMD gather instruction?. In *International Conference on Management of Data, DaMoN 2022, Philadelphia, PA, USA, 13 June 2022*. ACM, 9:1–9:5. doi:10.1145/3533737.3535089
- [10] Tomas Karnagel, Dirk Habich, and Wolfgang Lehner. 2017. Adaptive Work Placement for Query Processing on Heterogeneous Computing Resources. *Proc. VLDB Endow.* 10, 7 (2017), 733–744. doi:10.14778/3067421.3067423
- [11] Thomas Kissinger, Tim Kiefer, Benjamin Schlegel, Dirk Habich, Daniel Molka, and Wolfgang Lehner. 2014. ERIS: A NUMA-Aware In-Memory Storage Engine for Analytical Workload. In *International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures - ADMS 2014, Hangzhou, China, September 1, 2014*. 74–85. http://www.adms-conf.org/2014/adms14_kissinger.pdf
- [12] Roland Kühn, Jan Mühlhig, and Jens Teubner. 2024. How to Be Fast and Not Furious: Looking Under the Hood of CPU Cache Prefetching. In *Proceedings of the 20th International Workshop on Data Management on New Hardware, DaMoN 2024, Santiago, Chile, 10 June 2024*. Carsten Binnig and Nesime Tatbul (Eds.). ACM, 9:1–9:10. doi:10.1145/3662010.3663451
- [13] Chang Joo Lee, Veynu Narasiman, Onur Mutlu, and Yale N. Patt. 2009. Improving memory bank-level parallelism in the presence of prefetching. In *42st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-42 2009), December 12-16, 2009, New York, New York, USA*. ACM, 327–336. doi:10.1145/1669112.1669155
- [14] Sangjin Lee, Alberto Lerner, Philippe Bonnet, and Philippe Cudré-Mauroux. 2024. Database Kernels: Seamless Integration of Database Systems and Fast Storage via CXL. In *14th Conference on Innovative Data Systems Research, CIDR 2024, Chaminade, HI, USA, January 14-17, 2024*. <https://www.cidrdb.org/cidr2024/papers/p43-lee.pdf>
- [15] Baptiste Lepers, Vivien Quéma, and Alexandra Fedorova. 2015. Thread and Memory Placement on NUMA Systems: Asymmetry Matters. In *Proceedings of the 2015 USENIX Annual Technical Conference, USENIX ATC 2015, July 8-10, Santa Clara, CA, USA*. USENIX Association, 277–289. <https://www.usenix.org/conference/atc15/technical-session/presentation/lepers>
- [16] Feng Li, Sudipto Das, Manoj Syamala, and Vivek R. Narasayya. 2016. Accelerating Relational Databases by Leveraging Remote Memory and RDMA. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*. ACM, 355–370. doi:10.1145/2882903.2882949
- [17] Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. 2025. Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In *Proceedings of the 21st International Workshop on Data Management on New Hardware, DaMoN 2025, Berlin, Germany, June 22-27, 2025*. ACM, 2:1–2:9. doi:10.1145/3736227.3736231
- [18] Norman May, Daniel Ritter, Andre Dossinger, Christian Faerber, and Süleyman Sirri Demirsoy. 2023. DASH: Asynchronous Hardware Data Processing Services. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. <https://www.cidrdb.org/cidr2023/papers/p6-may.pdf>
- [19] Constantin Pohl, Kai-Uwe Sattler, and Goetz Graefe. 2020. Joins on high-bandwidth memory: a new level in the memory hierarchy. *VLDB J.* 29, 2-3 (2020), 797–817. doi:10.1007/S00778-019-00546-Z
- [20] Orestis Polychroniou, Arun Raghavan, and Kenneth A. Ross. 2015. Rethinking SIMD Vectorization for In-Memory Databases. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*. ACM, 1493–1508. doi:10.1145/2723372.2747645
- [21] Iraklis Psaroudakis, Tobias Scheuer, Norman May, Abdelkader Sellami, and Anastasia Ailamaki. 2016. Adaptive NUMA-aware data placement and task scheduling for analytical workloads in main-memory column-stores. *Proc. VLDB Endow.* 10, 2 (2016), 37–48. doi:10.14778/3015274.3015275
- [22] Scott Rixner, William J. Dally, Ujval J. Kapasi, Peter R. Mattson, and John D. Owens. 2000. Memory access scheduling. In *27th International Symposium on Computer Architecture (ISCA 2000), June 10-14, 2000, Vancouver, BC, Canada*. IEEE Computer Society, 128–138. doi:10.1109/ISCA.2000.854384
- [23] Lennart Schmidt, Johannes Pietrzyk, Juliana Hildebrandt, Alexander Krause, Dirk Habich, and Wolfgang Lehner. 2025. Rethinking MIMD-SIMD Interplay for Analytical Query Processing in In-Memory Database Engines. In *15th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 19-22, 2025*. <https://www.vldb.org/cidrdb/papers/2025/p7-schmidt.pdf>
- [24] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Gerry Fan, Yang Kong, Bo Wang, Jing Fang, Yuhui Wang, Tao Huang, Wenpu Hu, Jim Kao, and Jianping Jiang. 2025. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22-27, 2025*. ACM, 689–702. doi:10.1145/3722212.3724460